



Name https://t.me/arong_university

ID:

Section: C

Question 1 [15 Points]

Write a function named `replace_array` that would replace each element of a 2D array with the **magic number**. The **magic number** for an index (i, j) is defined as
magic number = (Sum of all elements in row i, excluding $A[i][j]$)

* (Sum of all elements in column j, excluding $A[i][j]$).

[You are not allowed to use any built-in functions `sum()`]

Sample Input:	Sample Output:	Explanation:																								
<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>9</td><td>8</td><td>7</td></tr><tr><td>2</td><td>8</td><td>1</td></tr></table>	1	2	3	4	5	6	9	8	7	2	8	1	<table><tr><td>75</td><td>84</td><td>42</td></tr><tr><td>132</td><td>180</td><td>99</td></tr><tr><td>105</td><td>240</td><td>170</td></tr><tr><td>126</td><td>45</td><td>160</td></tr></table>	75	84	42	132	180	99	105	240	170	126	45	160	Consider index (2, 1). The Sum of all elements in row 2, excluding $A[2][1]$ is $9+7 = 16$. The Sum of all elements in column 1, excluding $A[2][1]$ is $2+5+8=15$, so index (2, 1) will take the value of $16*15=240$. The process will be the same for all the indices.
1	2	3																								
4	5	6																								
9	8	7																								
2	8	1																								
75	84	42																								
132	180	99																								
105	240	170																								
126	45	160																								



CSE220: Data Structures (Lab)

Fall 2024

Lab Quiz - 02

Duration: 30 Minutes

B

Name: / https://t.me/arong_university	ID:	Section:
---	-----	----------

1) Imagine you're working with a **transaction processing system** where transactions are stored in a **singly linked list**. where each transaction is represented as a node in a **singly linked list**. Occasionally, duplicate transactions are entered consecutively due to a data sync issue. Your task is to clean up the transaction list by removing these consecutive duplicate transactions, ensuring that only unique transactions remain in the final list, regardless of how many times they were accidentally duplicated.

Modify the existing linked list(In-Place) and return the total amount of transactions removed.

Your task is to write a function `removeDuplicate(head)` / `removeDuplicate(Node head)`, that prints the sum of transactions removed and the modified list after removal of duplicates.

Sample Input	Sample Output
100→20→50→50→10→10→None	Total Duplicate Transaction Amount: 60 Modified List: 100→20→50→10→None



Name: https://t.me/arong_university	ID: ^	Section:
---	-------	----------

Question 1 [15 Points]

You are given a stack of integers. Write a function named **retain_and_reverse_stack()** that takes a stack st and an integer m as input parameters. The function modifies and returns the input stack such that:

1. Only the top m elements are retained in the stack (discard the rest).
2. The retained elements are reversed in the stack.

Assume the Stack class is already given and provides standard methods: push, pop, peek, and isEmpty.

Constraints:

- You are only allowed to use instances of the provided Stack class and its methods.
- No other data structures can be used apart from additional Stack instances.

Sample Input	Sample Output	Explanation
Stack: 10 20 30 40 50 60 n = 4	Stack: 40 30 20 10	Retain the top 4 elements (10, 20, 30, 40) and discard the rest (50, 60). Reverse the retained elements to get 40, 30, 20, 10.

https://t.me/arong_university



Name: _____

ID: _____

Section: _____

https://t.me/arong_university**Question 1 [15 Points]**

In this task, you are asked to implement a **HashTable** class that stores key-value pairs, where the key is a **string** (representing an employee ID) and the value is a **string** (representing the department name). The class should include a **hash_function** that computes the hash index based on the sum of ASCII values of each character in the key, sum each ASCII value by its position index (starting from 1) and finally sum these weighted values and take the modulus with the size of the hash table. The **insert()** method should insert a new key-value pair or update the value if the key already exists, using **forward chaining** to handle collisions. If the key already exists, its value should be updated. [You are not allowed to use any built-in functions except **len()**. Assume the display method is already implemented]

Sample Input:	Sample Output:	Explanation:
<pre>ht = HashTable(4) ht.insert("E123", "HR") ht.insert("BA", "Finance") ht.insert("XY", "Engineering") ht.insert("YX", "Marketing") print("\nHash table after insertions with collisions:") ht.display() ht.insert("E123", "Admin") print("\nHash table after update:") ht.display()</pre>	<pre>Hash table after insertions with collisions: Index 0: (XY: Engineering) -> (YX: Marketing) -> None Index 1: (E123: HR) -> None Index 2: (BA: Finance) -> None Index 3: None Hash table after update: Index 0: (XY: Engineering) -> (YX: Marketing) -> None Index 1: (E123: Admin) -> None Index 2: (BA: Finance) -> None Index 3: None</pre>	For E123: ASCII values: 'E' = 69, '1' = 49, '2' = 50, '3' = 51. Weighted sum: (1 + 69) + (2 + 49) + (3 + 50) + (4 + 51) = 70 + 51 + 53 + 55 = 229. Index: 229 % 4 = 1. For BA: ASCII values: 'B' = 66, 'A' = 65. Weighted sum: (1+66)+(2 + 65) = 134. Index: 134 % 4 = 2.

```
def hash_function(self, key):
    count = 1
    sum = 0
    for i in range(len(key)):
        sum += ord(key[i]) * count
        count += 1
    return sum % self.size
```

https://t.me/arong_university

Name: _____	ID: _____	Section: 01
-------------	-----------	-------------

Question 1 [15 Points]

A path in a binary tree is a sequence of nodes that connect a starting node to an ending node, where each consecutive pair of nodes in the sequence is connected by an edge in the tree. In a path, if you select any pair of nodes that are consecutive, the second node will be the child of the first node.

We will call a path an **upward** path if the elements in the nodes of that path form a **strictly increasing** sequence. In a strictly increasing sequence, an element **must be greater** than its previous element. $1 \rightarrow 2 \rightarrow 3$ is a strictly increasing sequence, $1 \rightarrow 2 \rightarrow 2 \rightarrow 3$ is increasing but not strictly.

In this question, you are given the **root** of a binary tree. Write a function `count_upward_paths(root)` that counts the number of paths that **start** from the **root**, **end** at a **leaf** node and are **upward** paths. Assume the tree is constructed, so write only this function.

- Assume the `BTNode` class is given. You cannot use any other data structures.
- No library functions allowed. You can write as many helper functions as you need.

Sample Input:	Sample Output:	Explanation:
Input Tree: <pre> graph TD 4((4)) --- 5((5)) 4 --- 6((6)) 5 --- 3((3)) 5 --- 9((9)) 3 --- 2((2)) 6 --- 8((8)) 6 --- 7((7)) 7 --- 1((1)) </pre>	Output: 2	The paths from the root to a leaf in the given tree are: $4 \rightarrow 5 \rightarrow 3 \rightarrow 2$ [not upward] $4 \rightarrow 5 \rightarrow 9$ [upward] $4 \rightarrow 6 \rightarrow 8$ [upward] $4 \rightarrow 6 \rightarrow 7 \rightarrow 1$ [not upward] Only the 2nd and 3rd paths form strictly increasing sequences. Therefore, only these two are upward paths.
Input Tree: <pre> graph TD 3((3)) --- 1((1)) 3 --- 2((2)) </pre>	Output: 0	There are no strictly increasing sequences formed by paths from the root to a leaf in this tree.

Name:	ID	Section:
-------	----	----------

Question 1 [15 Points]

In applications like batch task scheduling, it's vital to manage tasks based on their urgency, represented by priority levels where a lower number signifies higher urgency. You are given two arrays: one for task IDs and the other for corresponding priorities. These tasks must be organized using a data structure designed for efficient access to the most urgent tasks. However, only those tasks within a specified priority range, marked by `lowPriority` and `highPriority`, should be considered. Tasks falling outside this range should be excluded.

[Note: The most urgent tasks have lower priority numbers, so the data structure should focus on retrieving the minimum values within the range to correctly identify and process the highest-priority tasks]

Note:

https://t.me/arong_university

1. You cannot use any built-in functions except `len()`.
2. Direct sorting or searching methods on the array are not allowed.
3. Assume heap-related functions `extractMin()`, `sink()` are implemented and other necessary functions must be implemented.

Sample Input:	Sample Output:
tasks = [101, 102, 103, 104, 105, 106, 107, 108, 109] priorities = [9, 3, 7, 1, 5, 8, 2, 6, 4] lowPriority = 3 highPriority = 7	result = [102, 109, 105, 108, 103]
Explanation:	
Given lowPriority = 3 and highPriority = 7, tasks with the third, fourth, fifth, and sixth highest priorities based on the priority scale are selected. This corresponds to the tasks with priorities 3, 4, 5, and 6 after selecting them by their priority, which are tasks 102, 105, 101, and 108.	



CSE220: Data Structures (Lab)
Fall 2024
Lab Quiz - 07
Duration: 30 Minutes

B

Name: _____	ID: _____	Section: _____
-------------	-----------	----------------

Question [15 Points]

Write a Python or Java function called **reachingToFinalDestination**. The function takes the **given graph** represented as an adjacency matrix (a 2D list where $\text{graph}[i][j] = 1$ means there is a directed edge from node i to node j , and $\text{graph}[i][j] = 0$ means there is no edge from node i to node j). Also, the function takes a **destination** (the index of the destination node).

For the given graph, the function should return **True** if there exists any path (from **node 0** to the given destination **node 4**) and **False** otherwise.

Note: You have to manually build an adjacency matrix at first to represent the following directed unweighted graph. Also, you cannot use any built-in function, tuple, or dictionary. But you may build any helper function. And, assume that any code portion is not implemented.

Given Graph	Input of function	Output	Explanation
<pre> graph TD 0((0)) --> 1((1)) 1 --> 2((2)) 2 --> 3((3)) 3 --> 4((4)) </pre>	graph [as an adjacency matrix] destination [here, 4 as the index of the destination node]	True	The graph provided is a directed graph where node 0 can reach node 4 through the path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ (using 2D list traversal)

**Hint: Solving this question is much easier than getting out of bed this winter morning.*

https://t.me/arong_university